ALKADEMI - MAIN

AI NATIVE SOFTWARE ENGINEER BOOTCAMP

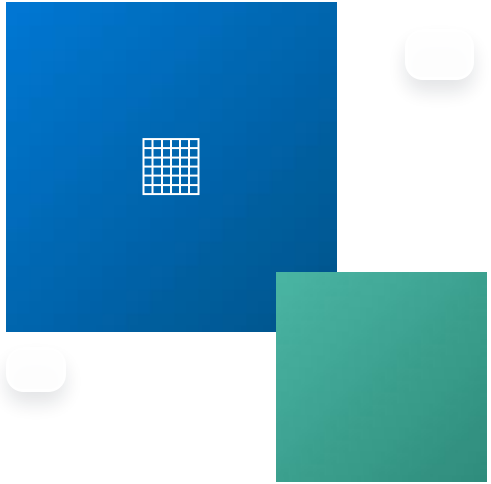
Connecting KelanaAI's Brain and Face

Build KelanaAI with Python, Next.js & Amazon Bedrock — Session 7

Duration: 2 Hours

Week 3 · Trip Dashboard

Feature: Trip History Dashboard



Yesterday KelanaAI learned how to interact with users.

Today it learns how to organize and present information.

SFIA Competency Card

Session 7 — Connecting KelanaAI's Brain and Face

FEATURE DELIVERED

✓ Trip History Dashboard

PRIMARY SFIA SKILLS

PROG L2 — Programming / Software Development

HCEV L2 — Human-Centred Design & User Experience

SWDN L2 — Software Design

TEST L1 — Testing

EVIDENCE — BY END OF SESSION, STUDENTS PRODUCE:

✓ **Multi-page Next.js application**
Routing, layout, and shared components across pages.

✓ **Trip History page**
Lists every saved itinerary as a clickable card.

✓ **Trip Detail page**
Dynamic route per trip — full itinerary view.

✓ **Frontend integrated with multiple FastAPI endpoints**
One page calls several backend services.

✓ **Git commit**
Versioned, reviewed, ready for the capstone repo.

`multi-page · services · components`

Learning Objectives

By the end of this session, students will be able to:

01

Navigate between pages in Next.js.

Multi-page apps need routing.

02

Retrieve data from multiple backend APIs.

One page can call several endpoints.

03

Display collections of data.

Lists and grids of trip cards.

04

Build reusable React components.

`<TripCard />` used many times.

05

Manage loading and error states.

Every page needs loading + empty + error.

06

Connect frontend to persistent backend data.

Read from PostgreSQL, not Bedrock.

Product Roadmap

Where we are — Week 3 begins, Trip Dashboard

WEEK 1 ✓	WEEK 2 ✓	WEEK 3 · TODAY	WEEK 4
✓ Trip Summary	✓ PostgreSQL	▶ Trip Dashboard	✓ Conversation History
✓ Recommendation Engine	✓ Amazon Bedrock	✓ Authentication	✓ Deployment
✓ REST API	✓ Next.js Frontend	✓ RAG	✓ Capstone

Yesterday KelanaAI learned how to interact with users. **Today it learns how to organize and present information.**

Story — Where Did Yesterday's Trip Go?

A traveler generates ten itineraries. KelanaAI only shows the latest. The rest are lost in the UI.

IMAGINE A TRAVELER...

...has generated ten different itineraries.

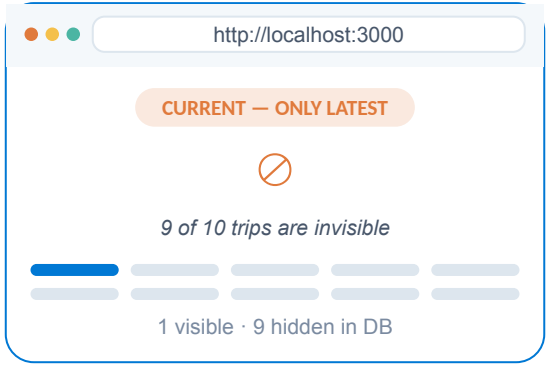
Today, KelanaAI only shows the latest result.

The other nine are in the database but invisible to the user.

? QUESTIONS TO CONSIDER

- ? How can users revisit an itinerary from yesterday?
- ? How can they compare multiple trips?
- ? How can they open a previous recommendation without regenerating?

Professional applications solve this with dashboards. Today we'll build KelanaAI's first.



The data exists in PostgreSQL — the UI just doesn't show it.

Technical Lesson

Eight parts — from app flow to a complete multi-page dashboard.

01 App Flow

02 Structure

03 Services

04 History Page

05 Dynamic Routes

06 Components

07 Empty State

08 Refresh

⚡Amazon Bedrock is not called every time — most loads read from PostgreSQL.

SESSION 07

Part 1 — Application Flow

Most user interactions retrieve existing data from PostgreSQL — Bedrock is only called on generation.

Architecture

Browser — User opens a page — 01

↓

Next.js — React frontend · routing — 02

↓

FastAPI — Python backend service — 03

↓

PostgreSQL — Persistent trip data — 04

↓

Amazon Bedrock — Only on generate — 05

Step 5 is rare. Steps 1–4 happen on every page load.

KEY INSIGHT

Don't call Bedrock every time

Browsing saved trips reads from PostgreSQL — fast and free. Bedrock is only invoked when the user generates a NEW itinerary. This keeps the app responsive and cost-efficient.

TWO READ PATHS

Browse trips	→ `PostgreSQL (fast, free)`
Generate trip	→ `Bedrock (slow, costs money)`

This separation is what makes the app usable at scale.

Part 2 — Project Structure

Organize the frontend into reusable folders. Large projects never put everything in one file.

Project Layout

```
frontend/

frontend/
└── app/                    # Next.js pages & routes
|   ├── page.tsx                # / (home)
|   ├── trips/
|       ├── page.tsx            # /trips (history)  NEW
|       └── [id]/page.tsx       # /trips/1 (detail) NEW
├── components/             # reusable UI  NEW
|   └── TripCard.tsx
├── services/               # API calls  NEW
|   └── tripService.ts
├── types/                  # TypeScript interfaces
└── lib/                    # utilities
```

New today: trips/ routes, components/, services/. Three new layers.

WHY STRUCTURE?

Separation makes apps maintainable

Each folder has one job. Pages render UI. Components are reusable blocks. Services handle networking. Types define data shapes. Lib holds utilities.

FOLDER JOBS

- **app/** pages & routes
- **components/** reusable UI
- **services/** API calls
- **types/** TypeScript shapes

This structure scales to any production Next.js app.

Part 3 — API Service Layer

Pages should not contain networking logic. Service files centralize all API calls.

Service Pattern

services/tripService.ts

```
// All trip-related API calls live here
const API_URL = "http://localhost:8000/api/v1"
export async function getTrips() {
  const res = await fetch(`${API_URL}/trips`)
  return res.json()
}
export async function getTrip(id: number) {
  const res = await fetch(`${API_URL}/trips/${id}`)
  return res.json()
}
export async function generateTrip(data: any) {
  const res = await fetch(`${API_URL}/trips`, {
    method: "POST",
    body: JSON.stringify(data)
  })
  return res.json()
}
```

Three functions — one per API endpoint. Pages import and call them.

WHY SERVICES?

Pages render. Services fetch.

Moving fetch() calls into service files keeps page components clean. If the API URL changes, you update ONE file — not every page.

BENEFITS

- ✓ Centralized API base URL
- ✓ Easy to mock for testing
- ✓ Pages stay focused on UI

USAGE IN A PAGE

```
import { getTrips } from
  "@/services/tripService"
const trips = await getTrips()
```

One import line per page — clean.

KELANAAI · SESSION 7

Environment Variables

Put URLs in .env – both frontend and backend

frontend/.env

```
# Frontend environment variables
# NEXT_PUBLIC_ prefix exposes this to the browser
NEXT_PUBLIC_API_URL=http://localhost:8000/api/v1
```

services/tripService.ts

```
// Read API URL from .env — no more hardcoding
const API_URL = process.env.NEXT_PUBLIC_API_URL

export async function getTrips() {
  const res = await fetch(`${API_URL}/trips`)
  return res.json()
}

export async function getTrip(id: number) {
```

NEXT_PUBLIC_ prefix is required — Next.js only exposes env vars with this prefix to the browser.

backend/.env

```
# Backend environment variables
# Frontend URL for CORS configuration
FRONTEND_URL=http://localhost:3000

# Database (from Session 4)
DATABASE_URL=postgresql+psycopg2://postgres:pass@localhost/kelana_ai
```

main.py

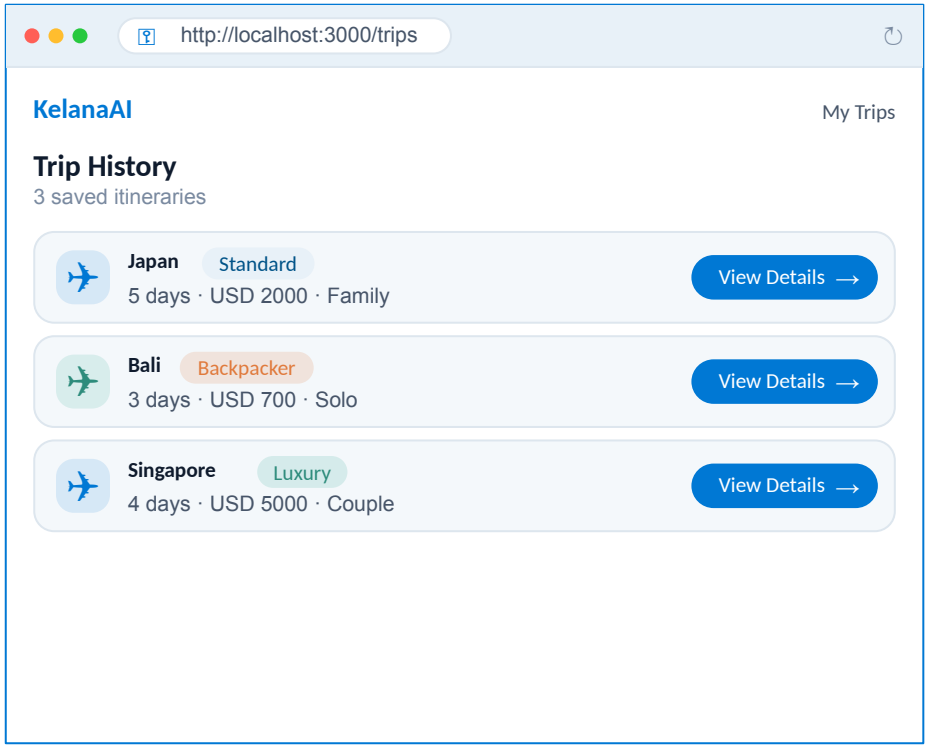
```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from dotenv import load_dotenv
import os

load_dotenv()
```

CORS reads FRONTEND_URL from .env — in production, just update .env to your Vercel URL.

Part 4 — Build Trip History Page

The /trips route. Calls GET /api/v1/trips. Renders each trip as a clickable card.

List View



app/trips/page.tsx

```tsx
import { getTrips } from "@/services/tripService"
export default async function TripsPage() {
  const trips = await getTrips()
  return trips.map(t =>
    <TripCard trip={t} />
  )
}
```

CONCEPT

Each trip = one card

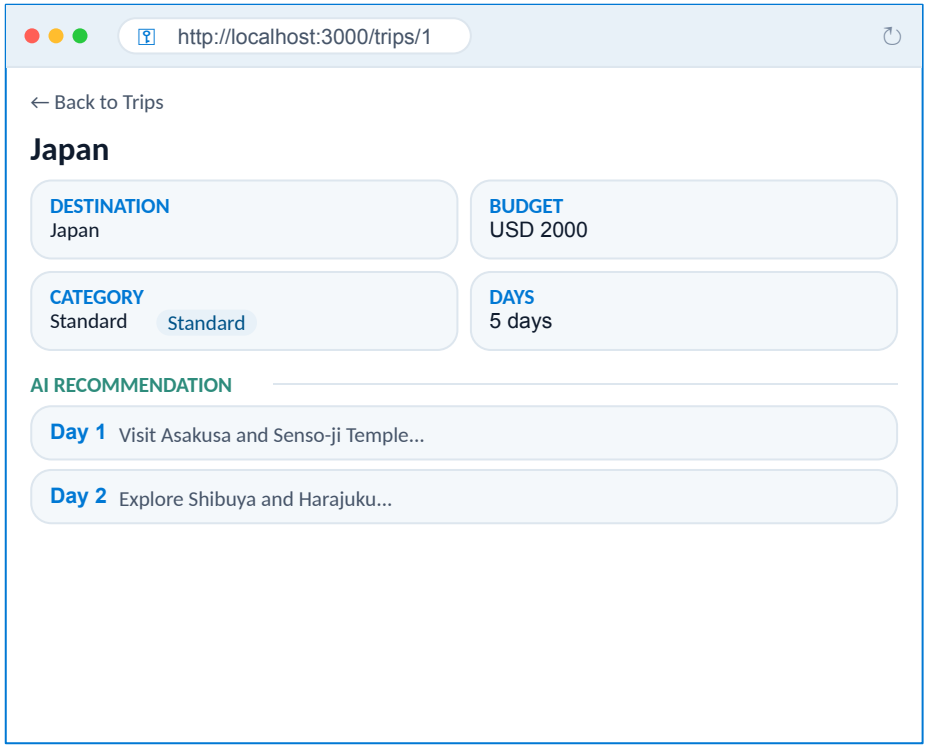
Call getTrips() once. Map the array into TripCard components. Each card links to /trips/[id] for the detail view.

Click 'View Details' → navigate to the detail page.

Part 5 — Dynamic Routes

Routing

The /trips/[id] route. One page template, many trips — the URL provides the ID.



DYNAMIC ROUTING

One template, many pages

The [id] in the folder name is a dynamic parameter. Next.js reads it from the URL and passes it to the page as a prop.

THE FLOW

1. User visits /trips/1
2. Next.js extracts id=1
3. Page calls getTrip(1)
4. FastAPI returns trip #1 from PostgreSQL

FILE STRUCTURE

```
app/trips/[id]/page.tsx
```

[id] = dynamic segment. Same template serves /trips/1, /trips/2, /trips/100...

Part 6 — Reusable Components

DRY Principle

Instead of repeating JSX, build once, reuse everywhere.

⊗ WITHOUT — REPEATED

```
// Same JSX, copied 3 times
<div>
  <h3>Japan</h3>
  <p>5 days · USD 2000</p>
</div>
<div>
  <h3>Bali</h3>
  <p>3 days · USD 700</p>
</div>
<div>
  <h3>Singapore</h3>
  <p>4 days · USD 5000</p>
</div>
```

Hard to maintain. Change a style → edit 3 places.

✓ WITH — REUSABLE

```
// Define once
export function TripCard({ trip }) {
  return (
    <div>
      <h3>{trip.destination}</h3>
      <p>{trip.days} days · {trip.budget}</p>
    </div>
  )
}

// Reuse anywhere
<TripCard trip={trip1} />
<TripCard trip={trip2} />
<TripCard trip={trip3} />
```

Change a style → edit ONE place. All cards update.

Reusable components improve consistency and reduce duplication.
This is the core principle of React.

Part 7 — Empty State

If there are no trips, show something helpful — not a blank screen.

UX Detail

http://localhost:3000/trips

BLANK — CONFUSING

Trip History

User sees nothing. Did the app crash? Are trips loading? No idea.

http://localhost:3000/trips

✓ HELPFUL — GUIDES USER

Trip History

No trips found.

Create your first itinerary.

Generate a Trip →

Friendly message + clear next step. Users know what to do.

Every application should handle empty data gracefully.
Empty states are an opportunity to guide users, not a dead end.

Part 8 — Refresh After Generation

End-to-End

After generating a trip, automatically redirect to the dashboard. The new trip appears at the top.

COMPLETE WORKFLOW

User fills form on /

↓

Click Generate → Bedrock creates itinerary

↓

Trip saved to PostgreSQL

↓

Auto-redirect to /trips

↓

New trip appears at top of dashboard

User never has to manually refresh. The app feels alive.

AUTO-REDIRECT

Generate → Dashboard

After the POST succeeds, Next.js redirects the user to /trips using useRouter().push(). The dashboard re-fetches and the new trip appears instantly.

CODE SNIPPET

```
import { useRouter } from "next/navigation"
const router = useRouter()
await generateTrip(data)
router.push("/trips")
```

WHY IT MATTERS

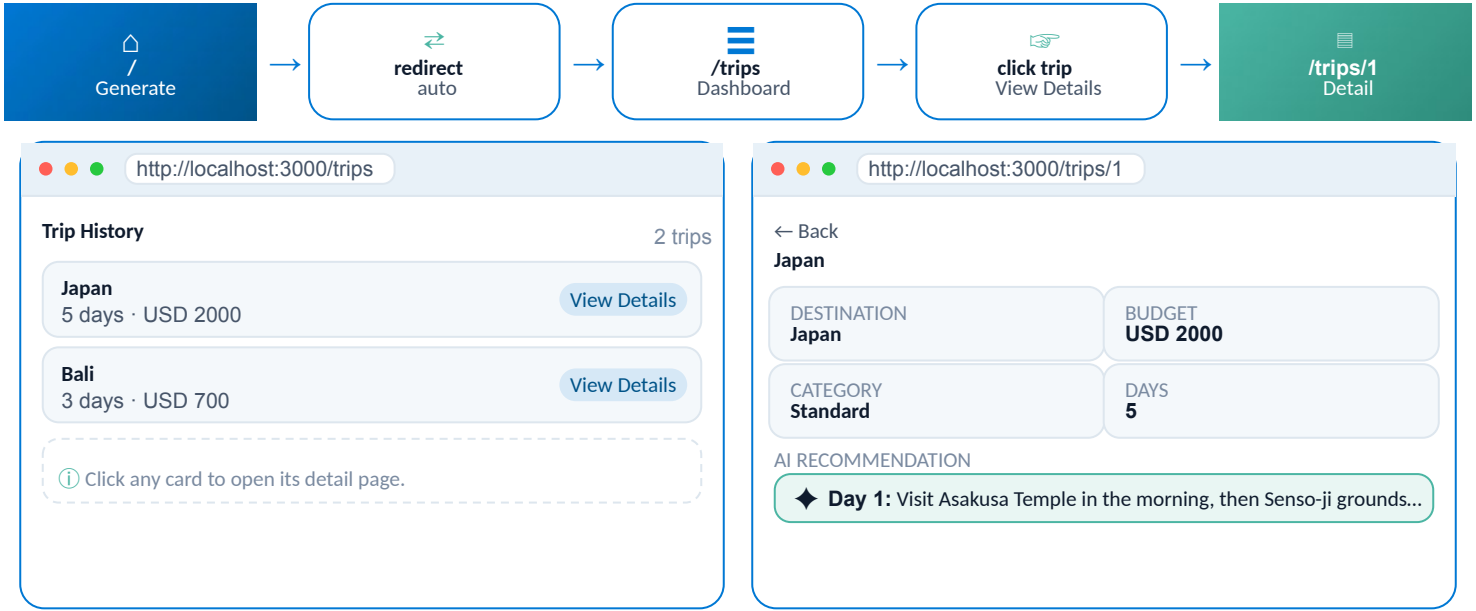
Students experience a complete end-to-end workflow. The app feels professional — no manual refresh, no confusion.

This is the moment KelanaAI feels like a real product.

Hands-on Lab — Multi-Page Flow

Build 3 pages wired end-to-end: `/` → `/trips` → /trips/1

30 min lab



Build all 3 pages. Wire the redirect. Verify clicking a trip opens its detail page.

Time-box: 30 min

Challenge

Add search and sort to the dashboard.

25 min total

CORE CHALLENGE — SEARCH

Add search functionality

Users can search trips by destination or travel style.

SEARCH BY:

- ✓ **Destination** (text search)
- ✓ **Travel Style** (text search)

UI HINT:

```
<input placeholder="Search trips..." />
```

Filter the trips array on every keystroke.

BONUS

Sort Trips

Let users sort the trip list by different criteria.

SORT BY:

- → **Latest** (newest first)
- → **Oldest** (first trip first)
- → **Highest Budget** (descending)

CODE HINT:

```
trips.sort((a, b) =>
  b.budget - a.budget
)
```

Use a dropdown to switch sort modes.

Git Checkpoint

Commit today's work. KelanaAI transforms from a single-page prototype into a multi-page application.

bash — kelana-ai

```
# stage today's dashboard work
$ git add .
# commit with a descriptive message
$ git commit -m "Create trip dashboard"
# push to GitHub
$ git push
```

Today transforms KelanaAI from a single-page prototype into a multi-page application.

Tag your commit: git tag session-7

Mini Quiz

Quick check — discuss with your neighbour, then we'll review.

4 questions

Q1

Why should API calls be moved into service files?

Hint: Think about what happens when the API URL changes.

Q2

What is a reusable component?

Hint: Define once, use many.

Q3

Why do we use dynamic routes?

Hint: One template, many items.

Q4

Why shouldn't Amazon Bedrock be called every time a page opens?

Hint: Speed and cost.

Discuss for 5 minutes. We'll review answers together.

Homework & What's Next

Polish the dashboard tonight — and tomorrow, KelanaAI learns to recognize users.

Homework — Enhance the Dashboard

Make trip cards richer and more informative.

01 **Destination icons**
Add a flag or landmark icon per destination.

02 **Budget formatting**
Display USD 2,000 instead of 2000.

03 **Category badge**
Color-coded: Backpacker / Standard / Luxury.

04 **Travel style badge**
Show Family / Solo / Couple on each card.

Bonus: pagination when > 10 trips.

↑ Commit and push your changes.

Next Session — 8

KelanaAI Learns Who You Are

Today KelanaAI became a complete multi-page app. Users can create, browse, and revisit trips. However...

"Everyone shares the same dashboard. There is no concept of personal accounts or private trips."

Tomorrow we fix that with authentication:

→ Implement JWT authentication
→ Each traveler gets a secure account
→ Personal travel history per user

Your trips become private. Your data becomes yours.

Milestone Achieved

KelanaAI became a real web application

Users can now browse previous itineraries, view detailed recommendations, and navigate between pages without generating new AI responses.

Multi-Page App

Navigation between `/`, `/trips`, `/trips/[id]`.

Service Layer

API calls centralized in `services/`.

Reusable Components

`<TripCard />` used across pages.

DB-First Reads

Browsing reads PostgreSQL, not Bedrock.

Business logic in the backend. Presentation in the frontend. Networking in service layers. Reusable components keep apps maintainable.

These practices prepare KelanaAI for its next step: supporting multiple authenticated users.